

Optimisation of the Roberta Model in Detecting AI-Generated Corporate Phishing Emails

Abstract

This dissertation investigates whether RoBERTa can be fine-tuned to detect AI-generated phishing emails using everyday computing power. It begins by reviewing recent literature on AI-generated text detection, phishing datasets and model robustness. Then, it develops a training pipeline using several labelled email datasets of increasing difficulty. The study benchmarks the base RoBERTa model, sequentially fine-tunes it across datasets, and evaluates performance using accuracy, F1 score, confusion matrices and model interpretability tools. Results show that the base model performs moderately well on email data, but fine-tuning substantially improves classification confidence and detection capability. However, performance is strongly affected by dataset quality, class balance, and distribution shifts, with the final stage showing a drop in accuracy when spam is introduced as an additional class.

Table of Contents

Abstract	1
Table of Contents.....	2
List of Abbreviations.....	4
1. Introduction	5
1.1 Background and Context	5
1.2 Problem Statement	6
2. Statement of Objectives.....	8
2.1. Scope	8
2.2 Research Aim.....	8
2.3 Research Objectives.....	8
2.4 Research Questions	9
2.5 Deliverables.....	9
2.6 Dissertation Outline	9
3. Literature Review.....	11
3.1 Introduction	11
3.2 Summaries	12
3.2.1 Decoding the AI Pen.....	12
2.2.2 The Phish, The Spam, and The Valid: Generating Feature-Rich Emails for Benchmarking LLMs	13
2.2.3 Can AI-Generated Text be Reliably Detected?.....	14
2.2.4 Detecting AI-Generated Text: Factors Influencing Detectability with Current Methods	15
2.2.5 Evaluating spam filters and Stylometric Detection of AI-generated phishing emails. Expert Systems with Applications.....	16
2.2.6 AI-Driven Phishing Email Detection: Leveraging Big Data Analytics for Enhanced Cybersecurity	17
2.2.7 Research Conclusions.....	17
4. Method	19

4.1 RoBERTa.....	19
4.2 Data Preparation	20
4.2.1 Training data	20
4.3 Training.....	21
4.3.1 Sequential Training.....	21
5. Implementation	23
5.1 Introduction	23
5.2 Environment Setup	23
5.2.1 Configuration and Utilities	23
5.2.2 Dataset Preparation	25
5.3 Baselining Performance	26
5.3.1 Performance Metrics	26
5.4 Sequential Training.....	29
5.5 Analysis Methodology	30
6. Performance Evaluation.....	32
6.1 Results	32
6.2 Strengths and Limitations	41
7. Interpretation and Conclusions	42
7.1 Conclusions	42
7.2 Implications.....	44
8. Recommendations and Further Work	45
9. References	46

List of Abbreviations

- AI — Artificial Intelligence.
- API — Application Programming Interface.
- AUROC — Area Under the Receiver Operating Characteristic curve.
- BERT — Bidirectional Encoder Representations from Transformers.
- CEAS — Conference on Email and Anti-Spam.
- CSV — Comma-Separated Values.
- F1 — F1 score.
- FERC — Federal Energy Regulatory Commission.
- GPT — Generative Pre-trained Transformer.
- GPU — Graphics Processing Unit.
- IBM — International Business Machines.
- JSON — JavaScript Object Notation.
- LLM — Large Language Model.
- NLP — Natural Language Processing.
- SHAP — SHapley Additive exPlanations.
- SVM — Support Vector Machine.
- TREC — Text Retrieval Conference.
- URL — Uniform Resource Locator.
- XGBoost — Extreme Gradient Boosting.

1. Introduction

1.1 Background and Context

In today's society, AI can be used as a professional tool which can help provide efficiencies and insights but can also lead to controversy through eliminating the creative arts. Unfortunately, more sinister actors also use AI for other harmful means such as fraud through email and phishing scams. Usage of AI and LLMs in everyday life is increasing, as stated by the National Literacy Trust and the rate of adoption of the technology is outpacing regulation within professional settings. Arguably one of the most impacted areas is education, due to a significant number of students using popular generative AI chatbots such as ChatGPT to generate large portions of (and in some cases entire) assignments.

According to the 2025 Cisco Cybersecurity Readiness Index (Cisco, 2025), of the business leaders with Cybersecurity responsibilities that were spoken to, 86% say their organisations have experienced AI-related security incidents in the past 12 months. Furthermore, 52% report that employees do not fully grasp how malicious actors are using AI to enhance their attacks.

The report also states that malicious actors are using ever more sophisticated means in their attempts to access networks, including AI powered phishing and deepfakes, requiring responses that can verify identities quickly, accurately, and at scale. The

The Cisco Talos 2024 Year in Review (Talos, 2025) highlighted that identity was a key theme for security incidents, with threat actors targeting users' digital footprints for malicious purposes. Identity attacks accounted for 60% of incidents recorded by the Cisco Talos Intelligence Team. The importance of identity attacks to malicious actors are that it allows them to facilitate other phases of their own operations via initial compromise and subsequent escalation.

The problem is further exacerbated by the growth in Data centre capacity that is expected to grow by 19-22% annually until 2030 which is fundamentally driven by AI workloads. This surge in traffic presents new security challenges as threat actors use AI to automate phishing, create deepfake social engineering attacks, and deploy adaptive malware. As such, identifying and preventing phishing emails will be a core element of the Cybersecurity toolbox to prevent identity theft and subsequent escalation.

Figure 1 below is a table taken from the 2025 Cisco Cybersecurity Readiness Index report (Cisco, 2025) and shows that identity is the most challenging cyberattack to protect against. This is partly due to the issues mentioned before that the majority of employees do not know how AI is being used to perform this identity theft. As such, this increases the need to minimise employee input into the identification process.

Areas companies find most challenging to protect against cyberattacks

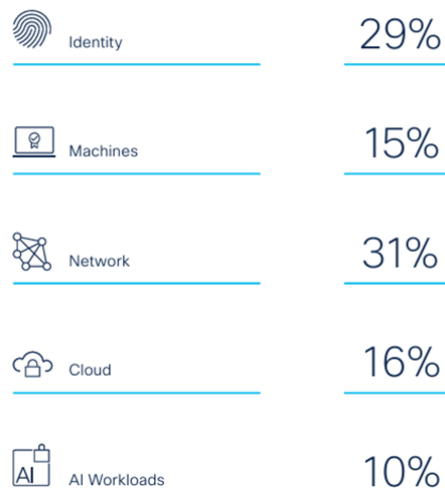


Figure 1: a ranking constructed by Cisco of which areas companies struggle to defend against cybercrimes

In an effort to combat the growth in phishing and the impacts, AI generated text detection has been the focus of lots of research by both universities and large corporations.

1.2 Problem Statement

IBM's annual Cost of a Data Breach Report 2025 (IBM, 2025) reported that the global average data breach cost USD 4.44 million. This cost incorporated both the cost of the stolen losses as well as any Government regulatory fines. IBM stated in the report that in some areas of the world, the cost per breach was actually lower when compared to 2024, but also noted that 16% of breaches reportedly involved bad actors using AI to develop phishing and deepfake attacks

Generative AI has demonstrated the ability to allow attackers to produce convincing emails that could be used by bad actors to manipulate victims into giving them access via phishing campaigns. IBM noted that gen AI reduced the time needed to craft a convincing phishing email from 16 hours down to only five minutes. 2025's report (IBM, 2025) shows the impact: on average, 16% of data breaches involved attackers using AI, most often for AI-generated phishing (37%) and deepfake impersonation attacks (35%).

Figure 2 below (IBM, 2025) shows the top attack vectors used by bad actors plotted against the actual cost of the attack. This shows that phishing was the most used

attack methodology and one of the costliest to organisations. This is no doubt driven by the pace at which these attacks can be developed as previously mentioned.

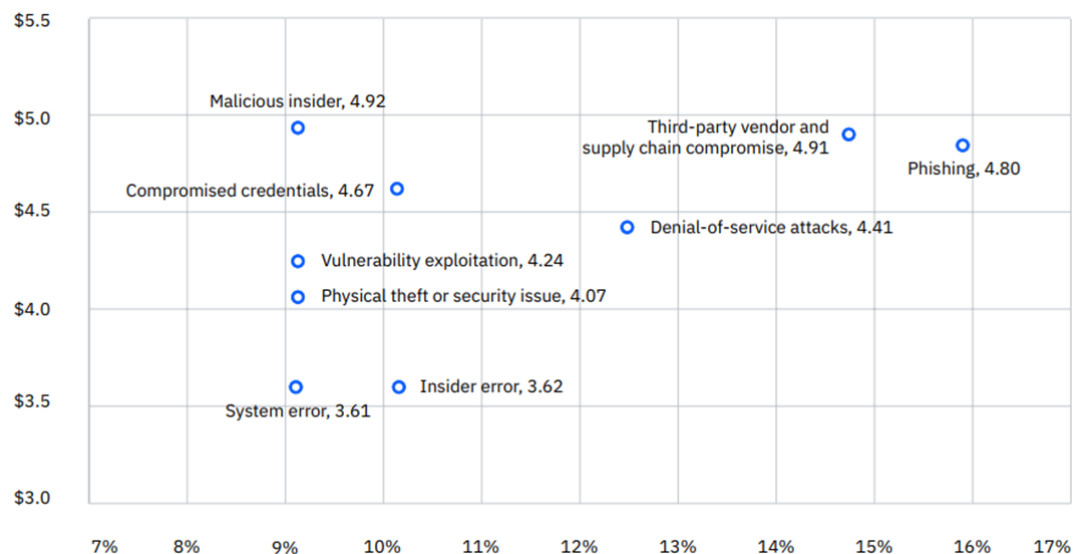


Figure 2: A graph by IBM showing the top attack vectors used by bad actors plotted against the cost of the attack

AI email phishing is a successful tactic as it provides LLMs access to public information from social media, previous data breaches and other open-source intelligence such as company websites to make the information within the email more convincing. This could be by looking for a Sales Order number or by asking employees to comment on a LinkedIn post or similar. (Kaspersky, 2026)

These corporate based attacks can often focus on high-ranking members of the company such as the CEO or the board of directors which is commonly known as “whale phishing”. Attacks focusing on mid-level managers are often referred to as “spear phishing”.

One of the more sophisticated examples of “whale phishing” was an incident in 2024 where an employee located in Hong Kong of the firm ARUP was invited via email to attend a video call. Unbeknown to the employee, every other participant on the call was a deepfake video and voice clone of their actual colleagues. During the call he was asked to transfer £20m to various accounts. (Milmo, 2024).

Developing adaptive, agile, low-cost implementation of black box techniques that can spot AI Generated text within emails is essential to minimise the impacts of data breaches to organisations and individuals. Through keeping costs low, this should enable organisations of all sizes to protect themselves and individuals.

2. Statement of Objectives

2.1. Scope

This research paper focuses on utilising a LLM – in this case the RoBERTa (Robustly Optimized BERT Pretraining Approach) model – measuring its baseline performance and then understanding how it can be trained and finetuned on challenging new datasets to classify emails as either phishing or legitimate. The baseline performance of the RoBERTa model will be measured as a reference point, and the model will be iteratively trained on ever increasing complex datasets and the performance compared.

Explainable AI tools will be used to provide insights as to what features the model uses to identify phishing emails and how these differ across the datasets used. The intention is to simulate real world implementation of the LLM as a black-box AI text detector that can be sufficiently trained and implemented with everyday computing power.

BERT (Bidirectional Encoder Representations from Transformers) was a model that was released by Google in 2018. This machine learning model was designed to understand the context of words in a sentence by looking at both the left and right sides of a word simultaneously. This was an advancement on prior models. However, RoBERTa has been chosen as the model within the research subject as it is an improved BERT. Research (RoBERTa: A Robustly Optimized BERT Pretraining Approach (Liu et al., 2019) showed that BERT could be optimised to outperform every other model released after it.

2.2 Research Aim

The aim of this research is to demonstrate the ease with which the RoBERTa model can be finetuned to improve its performance when detecting AI generated text within emails and suspected phishing.

2.3 Research Objectives

To achieve this aim, the study is guided by the following objectives:

- I. Review existing literature on utilising LLMs to detect AI generated text within large datasets with a focus on phishing emails and the associated dataset construction.

- II. Analyse the baseline performance of the RoBERTa model across complex datasets with varying degrees of difficulty.
- III. Develop an optimised RoBERTa model through iterative training on differing datasets and verify its performance through the process whilst providing insights on what features affect the performance.

2.4 Research Questions

This dissertation will attempt to answer the following questions:

1. What is the baseline performance of an untrained RoBERTa model across varying datasets?
2. Can the RoBERTa model be easily optimised to classify emails as Phishing?
3. What factors influence the accuracy of the model?
4. Are there limitations when trying to train RoBERTa using everyday computing power?

2.5 Deliverables

This research will produce a written report detailing the methodology, implementation and results of the study. The report will also include graphs, diagrams and screenshots of the code to better illustrate and convey the contents of the report. The report will also contain evaluations of the model's baseline performance when it is not trained, and this will be compared against its subsequent optimised performance. Explainable AI tools will be used to perform investigations into the features that drive the model to determine which text is AI generated.

2.6 Dissertation Outline

This dissertation is organised into eight main sections:

- Chapter 1 introduces the research background, problem statement, and motivation for using AI to detect phishing emails.
- Chapter 2 sets out the aim, objectives, research questions, deliverables, and overall structure of the work.
- Chapter 3 reviews the literature on AI-generated text detection, phishing datasets, and related model approaches.
- Chapter 4 explains the RoBERTa model, data preparation, and training process.
- Chapter 5 describes the implementation and baseline testing.
- Chapter 6 presents the evaluation results.
- Chapter 7 discusses the findings and their implications.

- Chapter 8 gives recommendations.

3. Literature Review

3.1 Introduction

This section will analyse existing research on current techniques and methodologies for identifying AI Generated text within emails. In addition to this, insights from the research will be sought to determine the type of datasets that may be suitable to be used to train the model. The research already existing around Phishing emails and machine learning will enable me to determine which label categories I should use, as well as areas where there is little or no focus, in order to maximize the potential contribution to the field of study this paper focuses on.

Table of Publications Reviewed

Title	Authors	Keywords
Decoding the AI Pen: Techniques and Challenges in Detecting AI-Generated Text (2024)	Abdali, S., Anarfi, R., Barberan, C. and He, J.	LLM; AI-generated Text Detection; Responsible AI; Watermarking; Paraphrasing Attacks; Data Poisoning
Constructing and Benchmarking: A Labelled Email Dataset for Text-Based Phishing and Spam Detection Framework	Rebeka Toth, Tamas Bisztray, Richard Dubniczky	phishing, spam, email security, large language models, email detection, emotion analysis, dataset
Can AI-Generated Text be Reliably Detected? (2024)	Sadasivan, V., Kumar, A., Balasubramanian, S., Wang, W. and Feizi, S.	AI-generated text detection, AI text detectors, LLMs, Recursive paraphrasing attack, Watermarked language models
Detecting AI-Generated Text: Factors Influencing Detectability with Current Methods. (2025)	Fraser, K., Dawkins, H. and Kiritchenko, S.	LLM, AI-generated text detection, Machine-learning detectors,
Evaluating spam filters and Stylometric Detection of AI-generated phishing emails. Expert Systems with Applications. (2025)	Opara, C., Modesti, P. and Golightly, L.	AI-generated phishing emails, Phishing detection, Stylometric analysis, Large Language Models (LLMs), Machine learning, Cybersecurity
AI-Driven Phishing Email Detection: Leveraging Big Data Analytics for Enhanced Cybersecurity	Bauskar S. R., Madhavaram C. R., Galla, E. P., Sunkara J. R., Gollangi H. K.	Phishing email detection, big data analytics, AI-driven, Cybersecurity, Machine learning

Table 1: a list of papers discussed as part of the literature review, with the title, authors and keywords for each paper discussed listed.

3.2 Summaries

3.2.1 Decoding the AI Pen

Decoding the Pen (Abdali et al., 2024) focuses on the potential misuse of Generative AI, in addition to a comparison of various methods of detecting AI-generated content. As LLMs (Large Language Models) are trained on extensive large datasets consisting of machine-readable text, they have become adept at producing human like text. To combat this, the paper details that researchers have developed detectors aimed at discerning between text authored by humans and that generated by AI. These detectors fall into two main categories – black-box methods and white-box detection.

Utilising a black-box detection approach, the detector has no access to the LLM that produced the AI generated text – it can only "see" the final text output. As such black-box detectors rely on collecting samples of both human-written and AI-generated text which are used to a classifier such as a supervised RoBERTa model. The trained classifier then discriminates between LLM-generated and human-written texts based on features extracted from the text samples. In effect, the black-box method almost relies on a stylistic estimation to determine if the text is AI generated or human generated.

White-box detection methods require access to the model that generated the text so that the logits are known. Logits are the raw, numerical scores generated within a Large Language Model (LLM) before they are turned into actual words. Because access to the LLMs inner workings is required for white-box detection, there are very few detectors currently available. As such black-box detection is more readily available as it is model agnostic and highly adaptable.

The research discusses five AI generated text detection techniques.

Supervised Detection is discussed as a detection approach where a model is fine-tuning on a dataset comprised of AI and human text. This is discussed as being inefficient as it is limited by high computational costs and the difficulty in attaining a dataset that is diverse enough. This also makes it vulnerable to data poisoning by attackers. This type of attack is severely impactful due to the usage of common large datasets by numerous AI detector tools currently in use.

The Zero-shot detection method is summarised. This method uses a pre-trained LLM to identify AI-generated text by utilising the fact that AI generated text follows a negative log probability curvature i.e. AI generated text follows a predictable path. By setting a low threshold along this curve, suspected passages of text can be flagged. This

methodology requires little set up time nor any large computational power as a LLM is used “straight out the box”. It is however still susceptible to spoofing and paraphrasing.

Retrieval-based detection is when a model searches a verified database for relevant knowledge in order to answer a question or perform any other task. This is mainly used to prevent hallucinations (where the model makes up references to something that doesn't exist). This also makes it vulnerable to data poisoning by attackers. This type of attack is severely impactful due to the usage of common large datasets by numerous AI detector tools currently in use.

Watermarking is a security measure used to identify AI-generated content, involving embedding a hidden signature or model-identifying feature directly into the AI's output. The signature is invisible to humans but can always be read by other models.

Feature-based is where an algorithm categorizes data into classes by analysing specific properties, known as features.

2.2.2 The Phish, The Spam, and The Valid: Generating Feature-Rich Emails for Benchmarking LLMs

Having an up to date, reliable and accurately labelled dataset to train LLMs for AI generated email text detection is the focus of this paper (Toth, Gruschka and Bisztray, 2026). Existing datasets have been found to be outdated whilst also lacking the separate distinct classes for phishing, spam and valid emails. Most widely used datasets in use today were gathered prior to 2010. An example is the Enron dataset which contains 1.6m emails and was made publicly available by the Federal Energy Regulatory Commission (FERC) in 2023. As such they lack the metadata and linguistic variance required for modern email classification.

A seed dataset was constructed from two different sources. Approximately 300 private emails were submitted voluntarily from personal and corporate enterprise inboxes. This source contained a balanced mix of spam, legitimate as well as phishing. Each email was annotated with subject, body, sender, all extracted URLs, and attachment filenames with extensions. A further 3000 emails were drawn from existing public datasets.

Five LLMs (Claude 3.5 Sonnet, GPT-5.2-Chat, Gemini-2.5-Flash, Qwen 2.5-7B-Instruct, DeepSeek-Chat) were benchmarked when identifying the primary explicitly requested user action (Follow the link, Open attachment, Reply and Unknown) in an email. Over five runs, the Strict Accuracy (defined as matching the ground truth labels defined by the two experts who assisted the research) was measured as well as Consistency (giving the same answer over each run).

Gemini-2.5-Flash achieved the strongest performance, reaching 97.98% strict accuracy with perfect internal consistency.

Parts of the legacy dataset had missing fields and intent (such as missing URLs) that needed to be populated. Five LLMs were assessed – Gemini 2.5 Flash, GPT5.2-chat, and Claude 3.5 Sonnet. The output was manually reviewed by two experts. Claude and GPT were found to frequently hallucinate placeholders, generating text like “[insert link here]” instead of a genuine URL based on the email’s narrative. Gemini-2.5-Flash consistently generated realistic, context-aware domains and filenames. This model was then used to label the remaining legacy emails in the dataset.

The seed dataset of 3300 emails was then expanded using LLM. For each template seed email, there were six synthetic variants. This produced an overall dataset of 23100 emails.

The research utilised Qwen-2.5-72B and Gemini-3.1-Pro to analyse the classification (Valid, Phishing, Spam) accuracy across four distinct data subsets across both Basic (Body and Subject) and Full (including metadata) configurations:

- Private Seeds (N = 300)
- Public Seeds (N = 3, 000)
- Private Variants (N = 1, 800)
- Public Variants (N = 18, 000)

On the basic setting, Gemini-3.1-Pro shows consistent performance across both Private and Public, but Qwen-2.5-72B performs worse on Private seed. Furthermore, adding metadata into the detection methodology improves the accuracy when looking at Private emails. It has limited impact on public emails. As such, it can be determined that metadata is essential for ensuring accurate detection.

When examining the accuracy against the individual classes, it was noted that the Qwen-2.5-72B model could only identify 91% of phishing emails whereas Gemini-3.1-Pro achieved an exceptional score of 96% with metadata incorporated.

It was noted that by incorporating metadata, each of the model’s inherent nature is altered and they become more “suspicious” by classifying more spam emails as valid. The trade-off is that more phishing emails are correctly identified.

2.2.3 Can AI-Generated Text be Reliably Detected?

This paper (Sadasivan et al., 2024) researches the effectiveness of four classes of AI generated text detectors that were introduced earlier in the section – watermarking based, neural network based (sometimes known as supervised learning), zero shot and retrieval based.

A paraphrasing attack is formulated to stress test each of the models. Paraphrasing is a methodology where a bad actor takes an AI generated response from a target model and then iteratively modifies the AI generated text to systematically avoid detection.

Three neural network-based paraphrasers were used to perform the attacks. The text quality was quantified using both human and automated methods. The conclusion was that the text quality diminished using the paraphrasers but was still sufficient to evade detection in settings such as a social media post. It would be more susceptible to detection when analysing an article.

To determine the accuracy, the AUROC (Area Under the Receiver Operating Characteristic curve) score is noted. AUROC provides a score between 0-1 and is defined as the probability that a model ranks a randomly chosen positive example higher than a randomly chosen negative one.

The AUROC score for a watermarked based model deteriorates from 0.99 for a passage with a token length of 50, to 0.43 after rounds of a paraphrasing attack. The success of the attack is lessened as the passage length (tokens) increases.

For a neural network-based detector, AUROC score degradation was witnessed across differing models. The RoBERTa-Large-Detector drops from 1 to 0.92 after the attack. It was noted that by allowing access to RoBERTa-Large-Detector to perform the paraphrasing, the score dropped even further to 0.8.

DetectGPT (Mitchell et al., 2023) was the Zero Shot model selected and when subjected to the paraphrasing attack saw its AUROC score drop from 0.97 to 0.60 after the attack.

A retrieval-based detector detailed in (Krishna et al. 2023) was chosen as it was designed to be robust against paraphrase attacks. However, the AUROC score degraded to 0.6 after 5 rounds of the attack.

Finally, they propose a theoretical link between the best achievable AUROC of any detector and the Total Variation distance between human and AI text distributions, arguing that as models get closer to human writing, reliable detection becomes fundamentally harder. Fundamentally this sets an upper boundary on the achievable AUROC score when mapped against the TV distribution.

2.2.4 Detecting AI-Generated Text: Factors Influencing Detectability with Current Methods

The paper (Fraser, Dawkins and Kiritchenko, 2025) categorises various classes of AI generated text. In arbitrary generation, the AI model has freedom for both the content and structure of the generated text. In guided generation, the user can guide the model on the general message or idea that should outputted. With controlled generation, the human control is exercised by the human inputting the text to be used and expressing

to the model the style of the text to be outputted for example. Collaborative class is where some text is AI generated and some is generated by the human or the AI generated text is altered by the human.

Current methods for detecting AI-generated text are systematically analysed and factors that make such text easier or harder to identify are considered. It distinguishes different detection approaches (watermarking, statistical and stylistic cues, and supervised detectors such as RADAR, Wild, T5Sentinel, Fast-DetectGPT, PHD, and LogRank) and tests them across a wide range of conditions, including different LLMs, languages, domains, text lengths, and levels of human obfuscation by attackers.

The paper shows that detector performance can vary dramatically depending on whether the generator model is known, whether the data are in or out of distribution, and whether users employ off-the-shelf evasion tools or partial human rewriting, often leading to severe drops in accuracy and reliability. Based on these findings, the paper provides practical recommendations, the best use cases for deploying detectors, concluding that real-world use requires caution and context awareness rather than blind trust in software accuracy numbers.

2.2.5 Evaluating spam filters and Stylometric Detection of AI-generated phishing emails. Expert Systems with Applications

The paper (Opara, Modesti and Golightly, 2025) investigates how well major email providers and stylometric methods detect AI-generated phishing emails created with GPT-4o. First the authors send 63 AI-generated phishing emails (and an equal number of legitimate AI-generated emails) through Gmail, Outlook, and Yahoo, finding that Gmail and Outlook allow more malicious messages to reach the inbox, while all three systems show notable false positives on benign AI emails. To go beyond traditional URL or domain-based defences, the authors built a text-only detector using 60 stylometric and linguistic features (47 of them new to phishing detection) and train four models (Logistic Regression, SVM, Random Forest, XGBoost) to distinguish AI phishing from legitimate AI messages based on style, structure, and tone. Results show that stylometric classifiers can accurately separate these closely related classes and highlight cues like imperative verbs, urgency, and clause density, offering a transparent and effective way to catch zero-day AI phishing that bypasses standard spam filters. Appendix A from the paper (Opara, Modesti and Golightly, 2025) presents the stylometric analysis, detailing the higher language complexity used in AI phishing emails compared to human-crafted legitimate emails.

2.2.6 AI-Driven Phishing Email Detection: Leveraging Big Data Analytics for Enhanced Cybersecurity

The paper (Bauskar et al., 2024) explains how combining AI-driven machine learning with big data analytics can significantly improve real-time phishing email detection and overall email security. It surveys the cybercrime and phishing landscape, reviews existing work on email security and big-data-based defences, and then motivates using large-scale email streams (e.g., Enron traffic) to train models that learn directly from raw data rather than handcrafted features, particularly to keep up with sophisticated, anti-forensic malware and social-engineering tactics. The paper proposes an AI/big-data architecture that integrates techniques such as anomaly detection, clustering, string matching, and NLP within large-scale infrastructures like Hadoop/NoSQL to detect ubiquitous phishing email abnormalities with minimal human intervention. They argue that this integration enables earlier and more accurate identification of phishing attempts, but also note practical limitations around adversarial attacks, data access, dataset labelling, and ethics, and highlight future directions like deep learning and few-shot learning for large security operations centres.

2.2.7 Research Conclusions

Most existing research around detection of AI-generated text does not focus specifically on phishing emails; research articles found by google scholar for the search term “using ai for detection of ai text” number at approximately 4.78 million. This is far more than the 11,000 results returned for the search term “using ai to detect phishing emails”.

This exact number is likely skewed due to the more generic keywords found in the first search. However, it does still demonstrate somewhat that Phishing detection-focused AI research is less prominent than well-known implementations such as Generative AI.

Several of the studies that were collated covered a multitude of models who all use differing methodologies. It was noticeable that both BERT and RoBERTa were mentioned in numerous studies, but specifically by (Sadasivan, V., Kumar, A., Balasubramanian, S., Wang, W. and Feizi, S., 2024) as having excellent detection capabilities. This was reassuring as the RoBERTa model forms the basis of this study.

An area for further development will be the ability of LLMs to become more resistant to recursive paraphrasing attacks. The report by (Sadasivan, V., Kumar, A., Balasubramanian, S., Wang, W. and Feizi, S., 2024) concluded that current general-purpose text detection software solutions are inadequate against such attacks. The

fact that such an accurate model as RoBERTa could be susceptible when the attacker is allowed access to the target model to perform the actual paraphrasing.

The research within the paper (Tóth, Bisztray, and Dubniczky's) focused on building and testing a database for the purpose of detecting AI-phishing. This is a useful paper as there was some excellent analysis of the impact and potential pitfalls of utilising a LLM to generate syntactic data. The dataset within the report will be used within this research to optimise the model due to its accessible format. While there are other papers that focus on AI-generated phishing, they were several years old and using out-of-date models.

The paper from (Bauskar S. R., Madhavaram C. R., Galla, E. P., Sunkara J. R., Gollangi H. K.) discussed the potential to leverage big data to implement phishing detection. Whilst this would undoubtedly provide excellent coverage, the purpose of this paper will be determining the feasibility of implementing and finetuning a RoBERTa model to detect phishing emails with everyday computing power.

4. Method

This section details the methodology that will be used to analyse the precise nature the effects fine-tuning has on a model with pre-existing knowledge when using a new dataset.

The model performance will be baselined and then iteratively trained on ever increasingly complex datasets. This is represented in the diagram below.

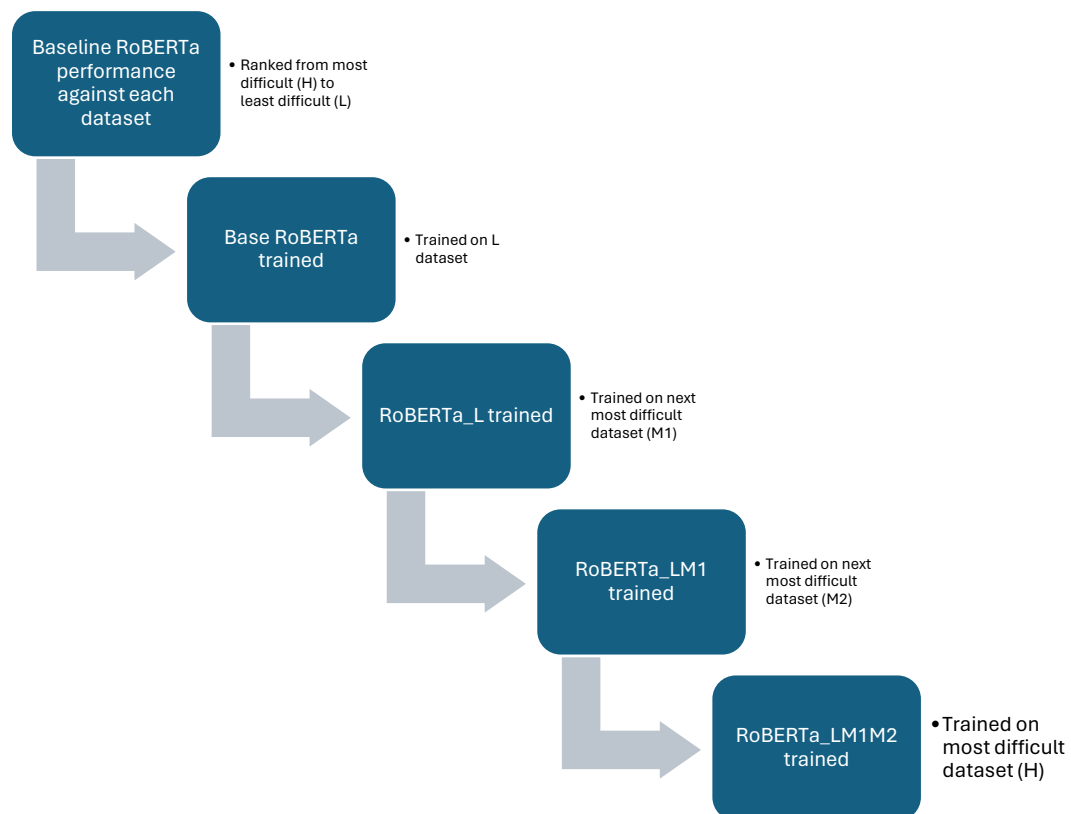


Figure 3: A diagram illustrating the sequential training process and the compounding experience it accumulates throughout

4.1 RoBERTa

The RoBERTa model is an encoder-only Transformer model, meaning it reads the full input text and builds contextual representations for each token (a token could be a word, sub-word or character) rather than generating text to one token at a time. Its backbone is a stack of Transformer encoder layers, and each layer refines the meaning of the tokens by combining attention and feed-forward processing. Below is a simple diagram of the overall RoBERTa architecture:

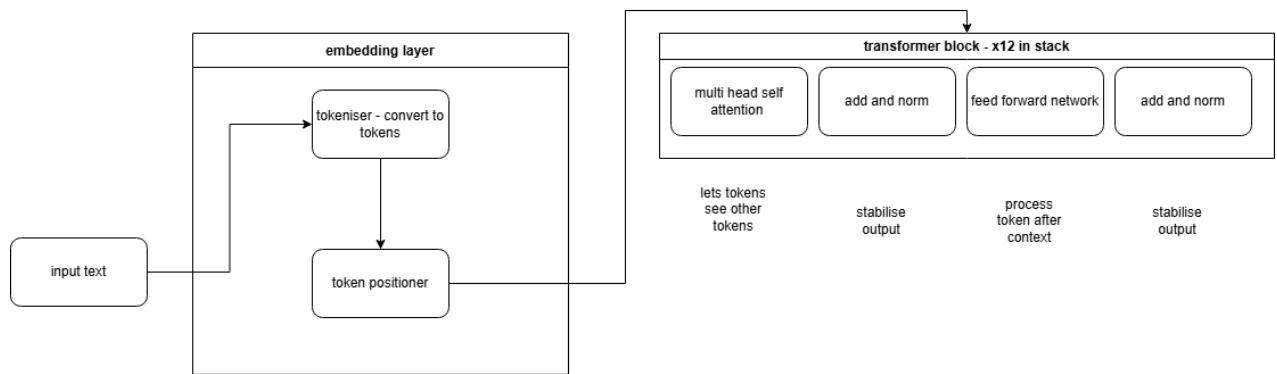


Figure 4: a simplified diagram illustrating the architecture of the RoBERTa model

The input first goes through a tokenizer, specifically byte-level, which splits text into sub-word pieces so the model can handle rare words and misspellings. Those token pieces are converted into token embeddings, which are learned vectors, and positional embeddings are added so the model knows token order. Special tokens are also inserted when needed to mark structure, such as the start of the sequence or separation between segments.

Self-attention is where the main classification occurs. In self-attention, each token compares itself with other tokens in the same sequence to decide what matters most for interpreting meaning. RoBERTa uses multi-head attention, which means it runs several attention mechanisms in parallel so different heads can capture different relationships at the same time.

After the classification, each encoder layer includes a feed-forward network, which is a small neural network applied to each token independently to further transform the representation. Residual connections add the input of a sublayer back to its output, which helps gradients flow during training and makes deep networks easier to optimize. Layer normalization is applied to keep activations stable and training more reliable. The output of many stacked layers is a set of hidden states, which are contextual embeddings: the vector for each token depends on the surrounding words, so the same word can mean different things in different sentences.

4.2 Data Preparation

4.2.1 Training data

Training any AIML model requires a sufficiently large and high-quality dataset. Datasets were primarily sourced from HuggingFace, Zenodo, and GitHub repositories found in research papers.

Four datasets were selected:

PhishFuzzer, from GitHub. This was the most recent dataset, and the first one selected due to the clear labelling of emails and extra data such as URLs, language, motivation provided along with the core information. However, it is also the smallest of the datasets selected at 3300 records. 3000 of these are taken from a Kaggle dataset and the SpamAssassin dataset. The remaining 300 are anonymised, taken from personal email accounts. While choosing other datasets, this one was changed several times by the author, with the latest posting also including a 19800-strong rephrased version, where each of the original 3300 emails was rephrased six times, with an additional field added to track which original email each rephrase belonged to. However, this change occurred after the 3300 set had already been baselined.

JinqiangDang, HuggingFace – this dataset was a collation of emails from other phishing and spam email datasets. While each email was labelled with the source dataset, manual combination of these various files into a single large dataset was required. The HuggingFace page for the set also contained parquet files for each split of the entire dataset.

MeAJOR, Zenodo – this was also a collection of existing phishing datasets, and used the same larger phishing and spam datasets, but differed in the fact it was the only set to use anonymised data. This meant that any names or other identifying information was replaced with “[NAME]”.

Evaldarob, HuggingFace – this was also a collation of other datasets however it also included data for phishing texts, URLs, websites in addition to emails. The email portion of the wider dataset is hosted on Kaggle. It was intended for this to be the fourth dataset in the training sequence, but each record only contained the body text and a phishing/valid label, so was ignored in favour of the PhishFuzzer dataset.

In order to train the model on these datasets, they must first be pre-processed into a format more readable for the model. In this case we will convert each dataset to JSON (JavaScript Object Notation) format.

4.3 Training

4.3.1 Sequential Training

The training of the model will use each dataset to iteratively finetune the RoBERTa model to better recognise phishing emails. The ranking produced when baselining performance will be the specific order that the datasets will be ran.

This training process was chosen to theoretically give the model a far greater understanding of phishing emails by increasing the amount of varied data the model would be exposed to.

Once the sequential training is complete the Captum PyTorch library will be used to analyse the performance of each individual finetuned instance against its predecessor to gain insights into the changed model behaviour.

5. Implementation

5.1 Introduction

This section details the implementation of the Method as described in Section 4. It will discuss the details of the baseline performance of the model and the datasets, as well as the code implementation of the training and the Captum explainer.

5.2 Environment Setup

5.2.1 Configuration and Utilities

The main part of setting up the environment was to configure variables such as google drive paths, label definitions and output the values to confirm they are correct. The first group of Colab cells contain utility functions such as email type normalisation to handle the variations in words used to label each email as Phishing and loading the dataset class. Colab is a web-based online platform for coding using cloud resources. This “setup” group of cells also contains the dataset class, which holds the core methods that tokenise the body text in each email batch that the model then predicts the labels for when training:

```
class EmailDataset(Dataset):
    """PyTorch Dataset for email classification"""
    def __init__(self, texts: List[str], labels: List[int], tokenizer, max_length: int):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        label = self.labels[idx]

        encoding = self.tokenizer(
            text,
            max_length=self.max_length,
            padding="max_length",
            truncation=True,
            return_tensors="pt",
        )

        return {
            "input_ids": encoding["input_ids"].flatten(),
            "attention_mask": encoding["attention_mask"].flatten(),
            "labels": torch.tensor(label, dtype=torch.long),
        }

print("Dataset class loaded")
```

Figure 5: A screenshot of the “EmailDataset” class that defines the tokenizer and fetches each email

Variable definition for batch size, file paths, and label names was done using a general config script:

```

import os
import torch

# google drive paths
CURRENT_STAGE = 2
DRIVE_PATH = "/content/drive/My Drive/Uni" # Google Drive root
MODELS_FOLDER = os.path.join(DRIVE_PATH, "models")
DATASETS_FOLDER = os.path.join(DRIVE_PATH, "datasets")
INPUT_FILE = os.path.join(DATASETS_FOLDER, "PhishFuzzer_emails_original_seed_v1.json") # Upload your file
OUTPUT_DIR = os.path.join(MODELS_FOLDER, "MattPhishingBERT_stage_4")

# model settings
MODEL_ID = "/content/drive/My Drive/Uni/models/MattPhishingBERT_stage_3_zenodo/checkpoint-29340"
MODE = "strict" # "strict" or "relaxed"
USE_SUBJECT = True
BATCH_SIZE = 16
LEARNING_RATE = 2e-5
EPOCHS = 3
MAX_LENGTH = 512
TEST_SIZE = 0.2
VAL_SIZE = 0.1
RANDOM_SEED = 42

# labels
STRICT_LABELS = ["Phishing", "Spam", "Valid"]
RELAXED_LABELS = ["Unwanted", "Valid"]

LABELS = STRICT_LABELS if MODE == "strict" else RELAXED_LABELS
NUM_LABELS = len(LABELS)

LABEL2ID = {lbl: i for i, lbl in enumerate(LABELS)}
ID2LABEL = {i: lbl for i, lbl in enumerate(LABELS)}

# column names - MUST MATCH JSON KEYS!!!
LABEL_COLUMN = "Type"
TEXT_BODY_COLUMN = "Body"
TEXT_SUBJECT_COLUMN = "Subject"
SOURCE_COLUMN = "Source"

# device
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

print("Configuration loaded")
print(f" Mode: {MODE}")
print(f" Device: {DEVICE}")
print(f" Labels: {LABELS}")
print(f" Input: {INPUT_FILE}")
print(f" Output: {OUTPUT_DIR}")
print(f"\nColumns:")
print(f" Label: {LABEL_COLUMN}")
print(f" Body: {TEXT_BODY_COLUMN}")
print(f" Subject: {TEXT_SUBJECT_COLUMN}")
print(f" Source: {SOURCE_COLUMN}")

```

Figure 6: A screenshot of the config script that defines the global variables

There is also a ‘diagnostic check’ cell for verifying the dataset is being read correctly that is used before commencing each stage of training. This check loads the database from the defined file path, then reads the first record to confirm if the label names match what is defined in the config cell. Then, it processes the first 5 records in the database using the utility functions (`normalize_category()`, `get_text()`, `detect_group()`). This, combined with a count of all labels in the database means that any faults in the logic will be caught before they can affect the training results. Below is an example ‘correct’ output using the MeAJOR dataset. The label name definitions are case sensitive, which is an easy mistake to make. Using the diagnostic script minimises time spent solving errors and prevents training being results being skewed by incorrect label reading.

```

=====
DIAGNOSTIC: Checking Dataset
=====

Loaded 108685 records

First record structure:
sender          : d66e9e64b006d6bca649f1c945129c42c4383687...
sender_domain   : enron.com
receiver        : 35c5a9fb9fba3b8737ed7cef2a87e427a73db4fc...
receiver_domain : enron.com
date            : 2001-06-29 09:37:04-05:00
subject         : [ORGANIZATION] failover plan.
content_types   : text/plain
body            : Hi [NAME],

Tonight we are rolling out...
urls            :
url_count       : 0.0
url_length_max  : 0.0
url_length_avg  : 0.0
url_subdom_max  : 0.0
url_subdom_avg  : 0.0
attachment_count : 0.0
has_attachments : False
attachment_types :
language        : en
source          : trec5
label           : benign

Testing processing on first 5 records:

Good Record 0: Valid      | TREC-05      | [ORGANIZATION] failover plan.
Hi [NAME],

Tonight we are ...
Good Record 1: Valid      | TREC-05      | RE: Intranet Site
[NAME] r these new?      | Intranet Site

[NAME]...
Good Record 2: Valid      | TREC-05      | FW: [ORGANIZATION] Company information
[NAME]/[NAME],

We a...
Good Record 3: Valid      | TREC-05      | New Master Physical
[NAME] and [NAME] -

Attached is a work...
Good Record 4: Valid      | TREC-05      | FW: [ORGANIZATION]/Mirant GISB
FYI. Below is a copy of my c...

```

```

Label distribution:
Phishing      : 48034 ( 44.2%)
Spam          : 0 ( 0.0%)
Valid         : 60650 ( 55.8%)
Unknown       : 1 ( 0.0%)

WARNING: 1 unknown labels found

```

Figure 7: An example output from the dataset diagnostic script

5.2.2 Dataset Preparation

Before undertaking the baselining and training, the data had to first be prepared. Most of the chosen datasets were originally in CSV file format, so they had to be converted to a format that was easier to understand and manage, such as JSON. JSON format is widely used due to being easy to read and manipulate for humans in addition to software. While the PhishFuzzer dataset is already in JSON format, the other datasets will need converting. Due to the massive amounts of data involved, Claude Haiku 4.5 (A smaller and faster variant of existing Claude models) was used via Visual Studio Code. It was instructed to convert the CSV files to JSON format, with each email being its own labelled JSON entry. It was also instructed to not change the content of the emails, only convert them to JSON format. In the new JSON format, each dataset shares several common fields:

- Sender – the sender of the message.
- Subject – the email’s subject line.
- Body – the main body text.

- Type – this is where the Phishing/Spam/Valid label is set, although some datasets use alternative categories such as “benign” for valid, or even 0 and 1 for phishing/spam and valid.

Some form of date and/or time is also included in all datasets, but not all have the complete date and instead include just the year. Other inconsistencies also appear with PhishFuzzer not having a receiver field, and whether each dataset separates the URL from the rest of the email body.

5.3 Baseline Performance

Once the datasets have been selected and converted to the desired format, the performance of the base RoBERTa model on these datasets had to be established. This was done by running the test script with the base RoBERTa model over each dataset and recording the output. The test script outputs two key metrics, accuracy and f1 scores. These statistics were used to determine which of datasets the base RoBERTa model found easiest and hardest.

5.3.1 Performance Metrics

In these tables below, N denotes the number of emails in the identified sub-group. Since not all datasets had included the original source of each email, the code defaulted to N being “Unknown” and having the F1 scores being for the whole dataset.

The F1 score is a metric used to evaluate the performance of a classifier model. It combines both precision and recall into a single metric, calculated using the following equation below:

$$F1\ Score = 2 \times \frac{recall \times precision}{recall + precision}$$

Recall is also known as the true positive rate (TPR). This is defined as the proportion of all actual positives that were classified correctly as positives. For example, when looking at the spam classification, recall would be measuring the fraction of spam emails that were correctly classified as spam. This is especially important as some of the datasets to be used are imbalanced so recall is a more meaningful metric than accuracy because it measures the ability of the model to correctly identify all positive instances within a class.

Precision is defined as the proportion of all the model's positive classifications that are actually positive. So as per above, if looking at spam class, precision measures the fraction of emails classified as spam that were actually spam. Precision and recall are each calculated using the following (Google for Developers, 2026a):

$$\text{precision} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

$$\text{recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negative}}$$

In the example of a spam email, a true positive (TP) is defined as a spam email that is correctly classified as a spam email.

In the example of a spam email, a false negatives (FN) is defined as a spam email that is misclassified as not-spam.

Accuracy itself is simply calculated with the number of correct predictions divided by the total number of email records.

This output gave us the following accuracies for each dataset when used with the base RoBERTa model:

Dataset Name	Accuracy (%)
MeAJOR	55.80%
JinqiangDang	53.60%
Zenodo curated combined	48.06%
PhishFuzzer	32.55%

Table 2: the accuracy of the base model over each dataset, taken from the results of the test script

The table shows the four datasets, sorted from easiest to hardest for the base RoBERTa model. Surprisingly, the smallest dataset (PhishFuzzer) was the hardest for the base model. The biggest dataset (MeAJOR) was the easiest for the model to classify, although it is unclear if this was an effect of the anonymisation of the data.

The baseline performance was established with the testing script, which loads the defined model (which in the case of identifying the baseline performance is the base RoBERTa model) and runs it over the entire dataset. The script uses the `predict_batch` function, which splits the dataset into batches and predicts the emails in each batch, moving onto the next batch and repeating until the dataset has been processed. In this implementation, the batch size was set to 16. This was to balance the memory usage and speed, as a larger batch size would use up compute units faster, of which Colab

users have a limited amount per month (however, more can be purchased at any time should they be needed). In the following table, the groups mentioned are existing Phishing email datasets from which samples were taken to make up PhishFuzzer.

Once finished, the test script outputs a percentage accuracy and a confusion matrix:

PhishFuzzer:

1126/3300 correct, accuracy 34.12%

Group	N	Accuracy	F1_Phishing	F1_Spam	F1_Valid
CEAS	839	0.00%	0.000	0.000	0.000
Nazario	1015	100.00%	1.000	0.000	0.000
SpamAssassin	1146	0.70%	0.014	0.000	0.000
manual	300	34.33%	0.511	0.000	0.000

Table 3: the results of the base model over the PhishFuzzer dataset, taken from the results of the test script

Zenodo dataset:

104451/217314 correct, accuracy 48.06%

N	Accuracy	F1_Phishing	F1_Spam	F1_valid
217314	48.06%	0.000	0.648	0.007

Table 4: the results of the base model over the Zenodo dataset, taken from the results of the test script

JinqiangDang:

107531/200609 correct, accuracy 53.60%

Note: an error in the code meant that for this dataset the labels indicating the original source dataset were not detected, causing the summary table to group all stats into one row. However, the accuracy percentage was the key statistic determining the ranking, so it was accepted.

N	Accuracy	F1_Phishing	F1_Spam	F1_Valid
200609	53.60%	0.000	0.000	0.698

Table 5: the results of the base model over the Zenodo dataset, taken from the results of the test script

MeAJOR:

60650/108684 correct, accuracy 55.80%

Group	N	Accuracy	F1_Phishing	F1_Spam	F1_Valid
TREC-05	49583	60.36%	0.000	0.000	0.753
TREC-06	15005	74.70%	0.000	0.000	0.855
TREC-07	44096	44.25%	0.000	0.000	0.614

Table 6: the results of the base model over the MeAJOR dataset, taken from the results of the test script

5.4 Sequential Training

The structure of Colab notebooks means that the original scripts need altering in several ways. The main change is separating functions such as dependency importing, variable definition, and the training and testing into their own cells. Utility functions such as the loading of the model and dataset also have their own cell to load them into the Colab session memory so they can be used later. Each cell in the Colab notebook holds one of the scripts, with each cell having its own output panel. Colab notebooks have their own area for saving file outputs, but integration with google drive can also be utilised.

The training script works in a similar way to the testing, where predictions are run in batches. The training script is designed to load the input model and database from google drive, run predictions and then output the newly trained model to the defined location (Google Drive in this instance). Where the training diverges from the test script is that the training takes place over several loops (known as epochs). For each epoch, you process emails in batches. In each batch, the model makes predictions on the emails and calculates:

- how wrong those predictions are compared to the true labels
- which weights caused those errors

The weights are then adjusted to reduce the error. After processing all batches in an epoch, the model evaluates itself on validation data to track whether it's improving. This process repeats for three epochs—each time the model processes all training data again, it adjusts the weights to try and improve. At the end of each loop the training and validation losses, the accuracy, and the F1 scores per email type are calculated and displayed. The output also produces a confusion matrix.

The training script was used to perform the sequential training cycle, where a model was trained on the previously described dataset ranking in order of easiest to hardest. At each stage, the data for the three epochs and the confusion matrix was recorded.

5.5 Analysis Methodology

In the analysis of the training results, I will use my understanding of overfitting and underfitting, applying it to the numerical results. I will also utilise the Captum and matplotlib libraries to better illustrate the comparison in behaviour between the base model and the trained version created by the sequential training. In addition, I will analyse the Confusion Matrices produced by each stage of the sequential training process. These Matrices are produced at the end of each stage and represent an evaluation of the new resulting model using the validation set of each stage's training dataset. As shown in the example below, a confusion matrix is a grid breaking down the judgements per category, making it easier to read the number of correct judgements per classification category. The grid spaces highlighted in green are the correct predictions, while the red-coloured spaces are incorrect predictions. Each grid space also contains a percentage for how much of the total sample number that type of judgement makes up (e.g. the 214 correct judgements make up 35.67 of the total judged emails.)

Example Confusion Matrix				
Predicted \ True	Phishing	Spam	Valid	SUM
Phishing	214 35.67%	2 0.33%	9 1.50%	225 95.11% 4.89%
Spam	5 0.83%	117 19.50%	33 5.50%	155 75.48% 24.52%
Valid	4 0.67%	11 1.83%	205 34.17%	220 93.18% 6.82%
SUM	223 95.96% 4.04%	130 90.00% 10.00%	247 83.00% 17.00%	536 / 600 89.33% 10.67%

Figure 8: An example confusion matrix

As previously mentioned, Captum will also be used to analyse the changes in the behaviour of Captum is a model interpretability library, built using PyTorch. It calculates statistics about which tokens and features had the most impact on a model's prediction. We will be using this functionality to give further insight into the changes made to the model by the sequential training beyond raw statistics on accuracy and loss.

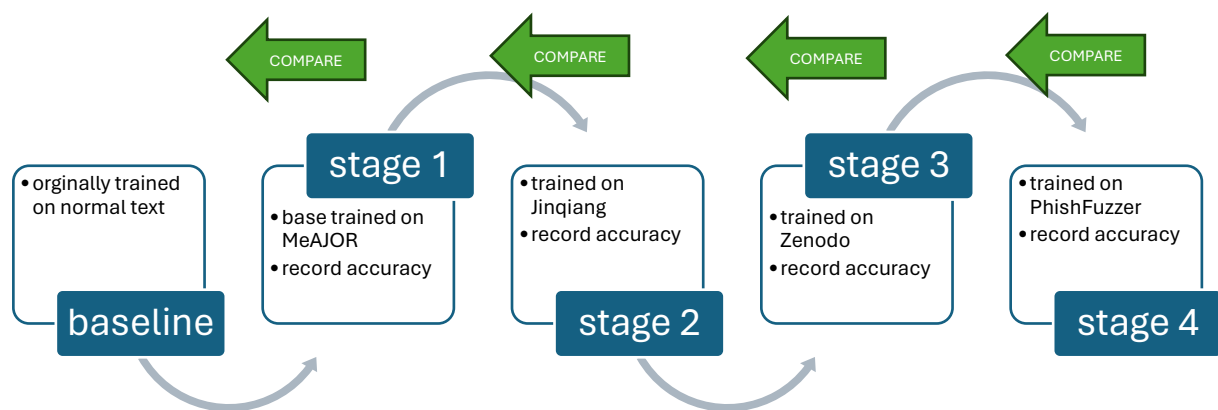


Figure 9: A diagram of the training process, illustrating how the study intends to compare the results

6. Performance Evaluation

6.1 Results

The overall results are as follows:

1. A fine-tuned model was successfully produced as a result of the sequential training.
2. Using Captum and matplotlib python libraries to plot bar charts, the trained model was shown to have a significant improvement in confidence when classifying emails.
3. In the last stage of the sequential training, accuracy drops significantly, most likely due to the final dataset having Spam as an additional email type.

The basic textual version of the matrix displayed by the script was converted into a colour-coded graphical version using a Confusion Matrix Generator (Perri, Simonetti and Gervasi, 2021). To keep the matrices consistent, the spam category has been kept for all stages despite only being in stage 4.

Reviewing the output data reveals that accuracy peaked at epoch three of stage 3, where the JinqiangDang model was training on the Zenodo dataset. After this, accuracy drops significantly in stage 4 when the output Zenodo model from stage 3 is trained on the PhishFuzzer dataset. This drastic drop in accuracy suggests that by the end of stage 3 the model had undergone major overfitting and failed to recognise patterns or writing styles in the final dataset.

A graph of the accuracies for each epoch in the training process was also produced by inserting the values into Excel and then using it to populate a line graph with data markers for each entry. The Y-axis represents the accuracy, while the X-axis shows (in order from top to bottom) the epoch number, dataset used, model used, and stage number.

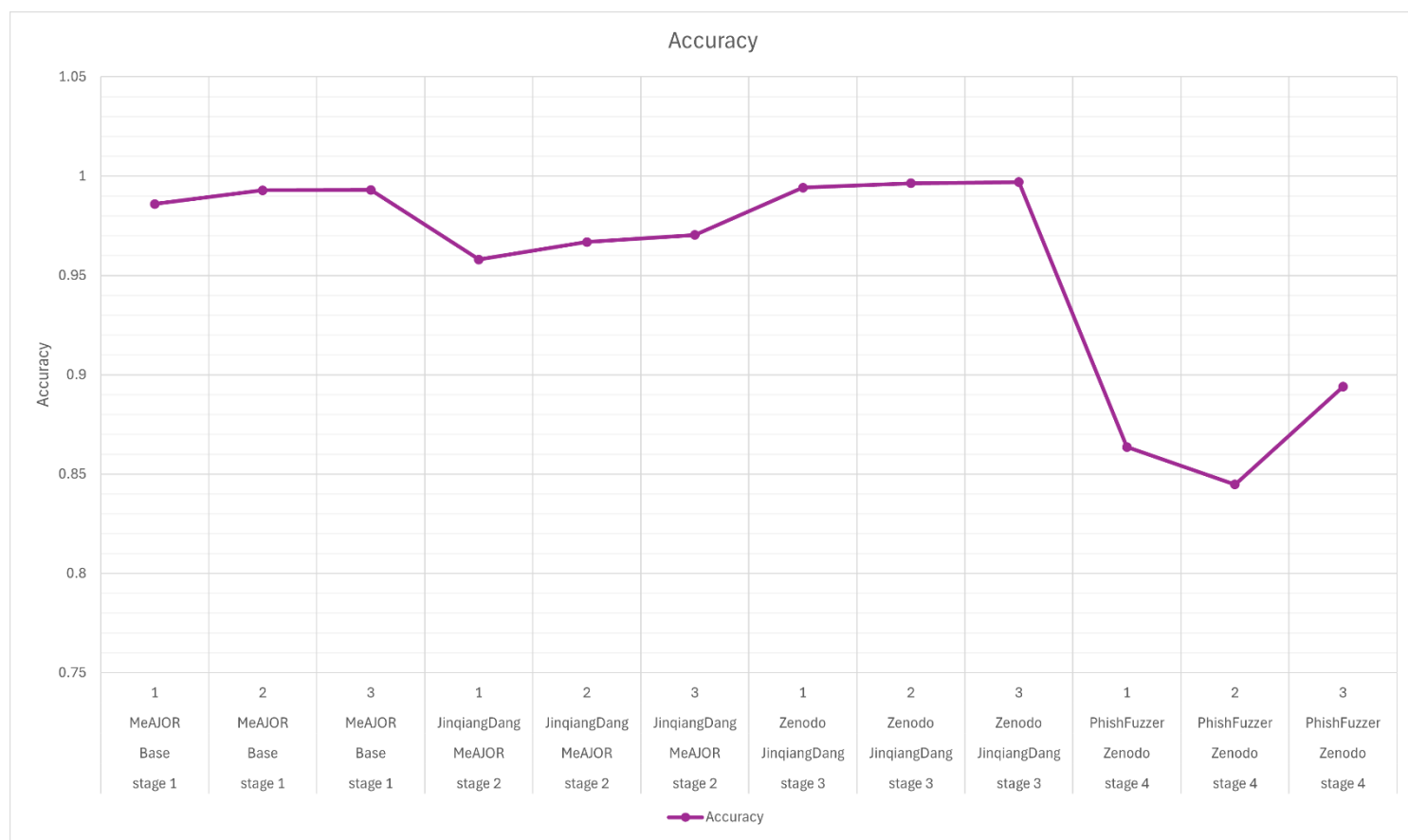


Figure 10: A line graph showing the changes in accuracy over the course of the sequential training.

The accuracy graph above shows that while the accuracy peaked at the end of stage 3, a slight reduction also occurs before in the first epoch of stage 2. This indicates that the model has undergone slight overfitting during stage 2. The stage by stage training logs show that the model could be too tightly bound to the patterns of the first three datasets. This could be due to the fact that the first three datasets could be possibly being more similar to each other than each of them was to the PhishFuzzer dataset. It can also be noted that PhishFuzzer was one of the two datasets to have Spam as a classification type.

Below is a loss graph, showing both training and validation loss throughout the entire sequential training process. The X-axis remains the same as the accuracy graph, but for this graph the Y axis now represents the validation (green) and training (orange) loss.

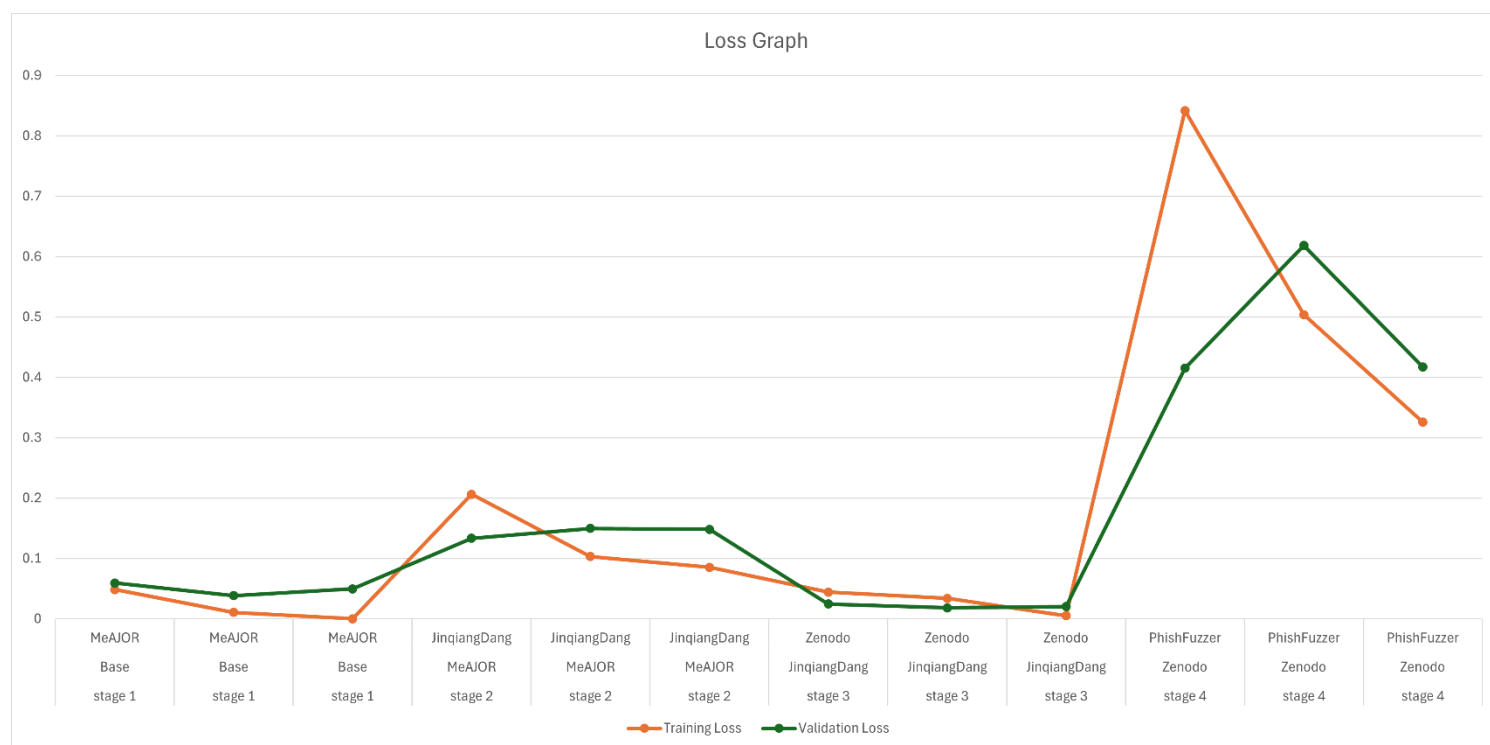


Figure 11: A loss graph for training process, created using the raw loss data

Stage 1, Base RoBERTa on MeAJOR dataset

Epoch	Training Loss	Validation Loss	Accuracy	F1 Phishing	F1 Spam	F1 Valid
1	0.048176	0.059045	0.985969	0.983952	0.000000	0.987536
2	0.010449	0.038437	0.992869	0.991948	0.000000	0.993602
3	0.000076	0.049610	0.993099	0.992171	0.000000	0.993831

Figure 12: The summary table from the end of training stage 1, showing the Loss, Accuracy, and F1 score

Base Model on MeAJOR Dataset				
Predicted \ True	Phishing	Spam	Valid	SUM
Phishing	9493 43.67%	0 0.00%	114 0.52%	9607 98.81% 1.19%
Spam	0 0.00%	0 0.00%	0 0.00%	0 0.00% 0.00%
Valid	51 0.23%	0 0.00%	12079 55.57%	12130 99.58% 0.42%
SUM	9544 99.47% 0.53%	0 0.00% 0.00%	12193 99.07% 0.93%	21572 / 21737 99.24% 0.76%

Figure 13: A confusion matrix showing the distribution of predictions between correct and the false positives and negatives

Stage 1's results show the performance of the base model on the MeAJOR dataset. This dataset was relatively unique due to the anonymization of the data, meaning that information such as names and addresses were replaced with square brackets containing the name of the information type that had been replaced. The base model performed surprisingly well on the dataset given that it was not trained on phishing emails, getting nearly all of its predictions right. As shown in the confusion matrix above, it performed 0.43% better on classifying Valid emails than it did Phishing emails.

Stage 2, MeAJOR model on JinqiangDang data:

Epoch	Training Loss	Validation Loss	Accuracy	F1 Phishing	F1 Spam	F1 Valid
1	0.206220	0.133235	0.958004	0.954162	0.000000	0.961251
2	0.103189	0.149507	0.966852	0.964141	0.000000	0.969181
3	0.085339	0.148227	0.970341	0.967894	0.000000	0.972441

Figure 14: The summary table from the end of training stage 2, showing the Loss, Accuracy, and F1 score

MeAJOR Model on Jinqiang Dataset				
Predicted \ True	Phishing	Spam	Valid	SUM
Phishing	17906 44.63%	0 0.00%	710 1.77%	18616 96.19% 3.81%
Spam	0 0.00%	0 0.00%	0 0.00%	0 0.00% 0.00%
Valid	637 1.59%	0 0.00%	20869 52.01%	21506 97.04% 2.96%
SUM	18543 96.56% 3.44%	0 0.00% 0.00%	21579 96.71% 3.29%	38775 / 40122 96.64% 3.36%

Figure 15: A confusion matrix showing the distribution of predictions between correct and the false positives and negatives

Stage 2 had the newly created MeAJOR-trained model train on the JinqiangDang dataset, which did not anonymise its data. The Jinqiang dataset was nearly double the size of the previous dataset, meaning there was a lot more training records to process. The confusion matrix above shows that the total number of correct judgements across both Phishing and Valid email types has decreased by 2.6%. While the immediate 0.005 decrease in accuracy is not the preferred result of that first epoch of the stage, the accuracy does in fact increase to 0.970 in the final epoch of this second stage.

Stage 3, JinqiangDang model on Zenodo data:

Epoch	Training Loss	Validation Loss	Accuracy	F1 Phishing	F1 Spam	F1 Valid
1	0.044065	0.024465	0.994248	0.993972	0.000000	0.994501
2	0.033740	0.018146	0.996376	0.996222	0.000000	0.996519
3	0.005061	0.020091	0.997009	0.996876	0.000000	0.997132

Figure 16: The summary table from the end of training stage 3, showing the Loss, Accuracy, and F1 score

Jinqiang Model on Zenodo Dataset				
Predicted \ True	Phishing	Spam	Valid	SUM
Phishing	20706 47.64%	0 0.00%	118 0.27%	20824 99.43% 0.57%
Spam	0 0.00%	0 0.00%	0 0.00%	0 0.00% 0.00%
Valid	52 0.12%	0 0.00%	22587 51.97%	22639 99.77% 0.23%
SUM	20758 99.75% 0.25%	0 0.00% 0.00%	22705 99.48% 0.52%	43293 / 43463 99.61% 0.39%

Figure 17: A confusion matrix showing the distribution of predictions between correct and the false positives and negatives

Stage 3 trains the JinqiangDang-tuned model on the Zenodo dataset, resulting in a further increase in accuracy, with the final epoch's accuracy increasing to above the base model's accuracy at 0.997. Over 2% more Phishing emails are being correctly predicted by the model, however Valid email prediction success rate has decreased by 0.003%. Epoch three of this stage sees training loss at its lowest point at 0.005, while validation loss increases slightly to 0.020 from 0.018.

Stage 4, Zenodo model on PhishFuzzer data:

Epoch	Training Loss	Validation Loss	Accuracy	F1 Phishing	F1 Spam	F1 Valid
1	0.841833	0.415235	0.863636	0.865385	0.838710	0.884848
2	0.503518	0.618272	0.844697	0.919786	0.802721	0.804124
3	0.325797	0.417265	0.893939	0.943820	0.876543	0.861702

Figure 18: The summary table from the end of training stage 3, showing the Loss, Accuracy, and F1 score

Zenodo Model on PhishFuzzer Dataset				
Predicted \ True	Phishing	Spam	Valid	SUM
Phishing	214 35.67%	2 0.33%	9 1.50%	225 95.11% 4.89%
Spam	5 0.83%	117 19.50%	33 5.50%	155 75.48% 24.52%
Valid	4 0.67%	11 1.83%	205 34.17%	220 93.18% 6.82%
SUM	223 95.96% 4.04%	130 90.00% 10.00%	247 83.00% 17.00%	536 / 600 89.33% 10.67%

Figure 19: A confusion matrix showing the distribution of predictions between correct and the false positives and negatives

The final stage trains the stage 3 model on the PhishFuzzer dataset. However, the model accuracy drops down to 0.893. This is likely due to the sudden introduction of Spam, which the model struggles to differentiate from Phishing. This, in combination with overfitting on the previous stage, means that accuracy is significantly lower and losses increase significantly as well.

The model interpretability library Captum was also used in a separate script to better analyse how the base model read the emails and what specific tokens had the most impact on the predicted classifications. Captum is a mathematics library used to calculate statistics on token impact on the model predictions. The bar graphs shown below demonstrate that while the base model could correctly identify a Valid email, it could not identify Phishing or Spam emails.

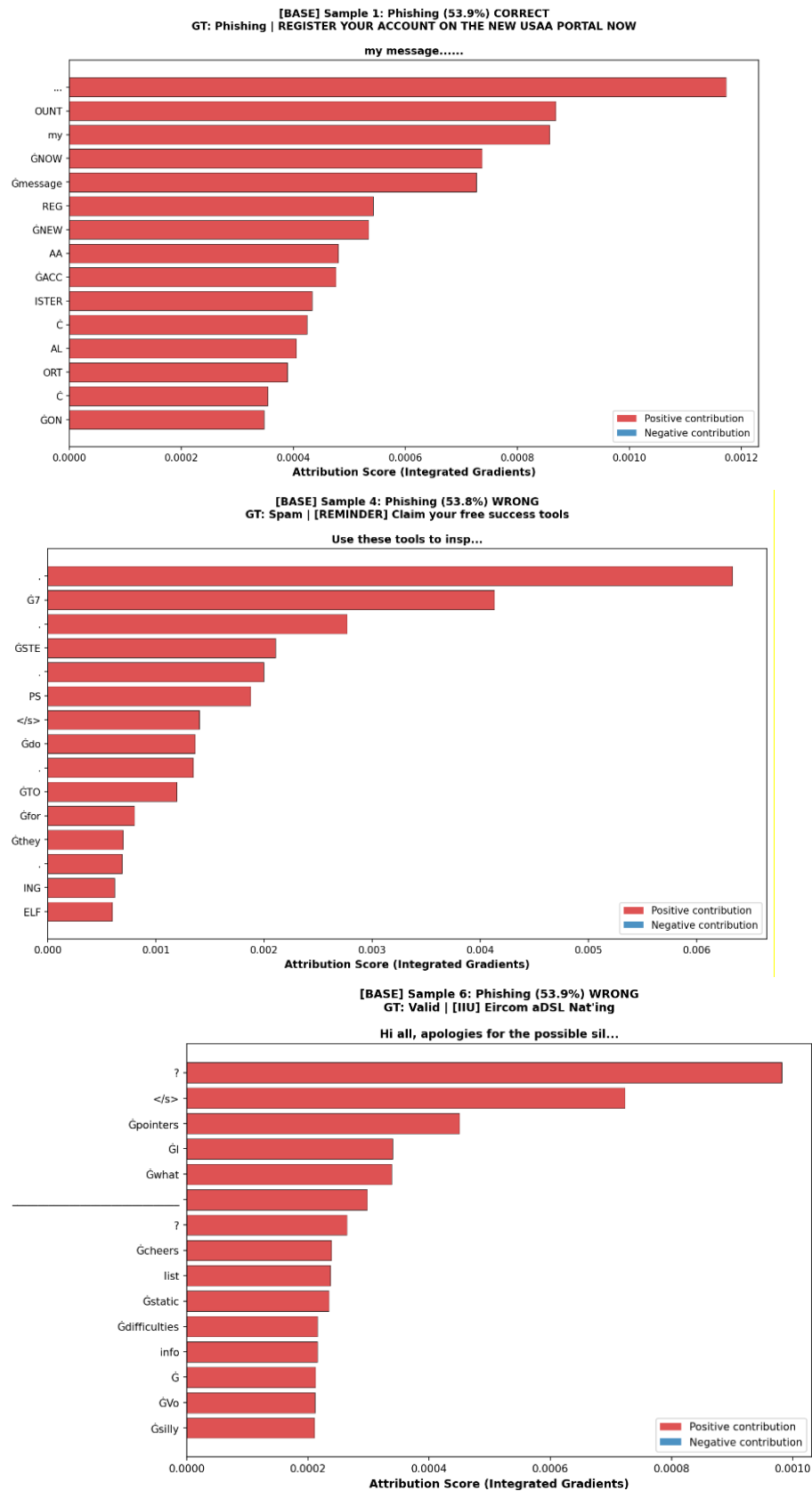
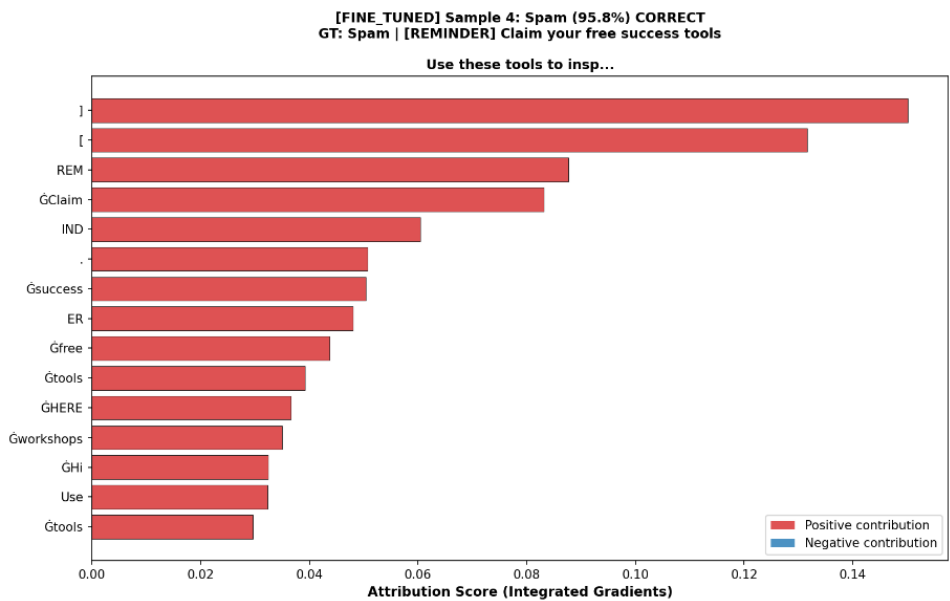
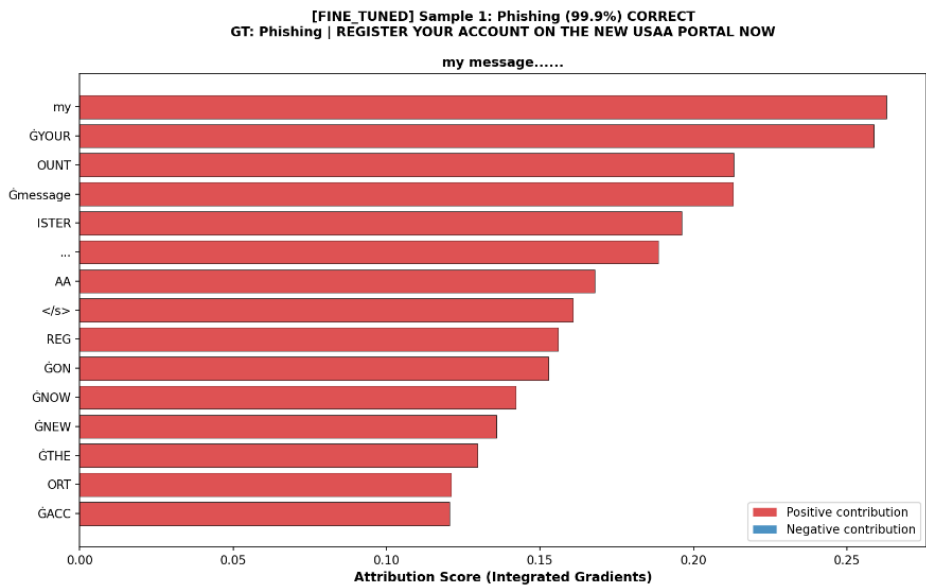


Figure 20: Bar graphs for the base model's predictions for a Phishing, Spam and Valid email respectively.

Bar graphs were also produced for the final trained model as shown below to compare it against the original base model:



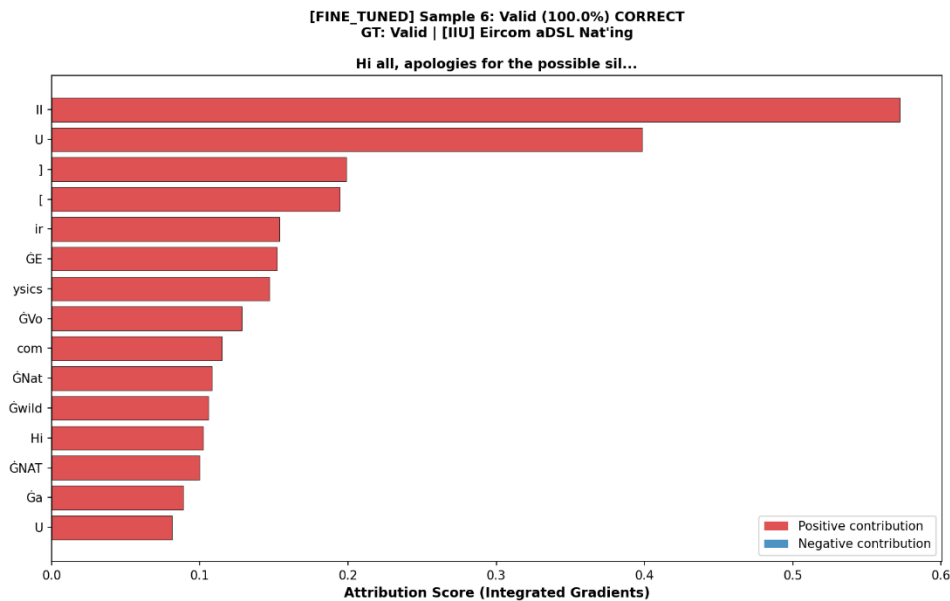


Figure 21: Bar graphs for the Final stage model's predictions for a Phishing, Spam and Valid email respectively.

6.2 Strengths and Limitations

Overall, I believe that my approach was thought out and was based on correct interpretations of existing machine learning research and other related literature in addition to other reliable non-academic sources.

However, the approach had several limitations. There were inequalities in the datasets, with the final stage using a dataset that contained Spam as an additional category. While it has decreased accuracy in stage 4 of the methodology, it was used in the first place due to significant difficulty in locating suitable labelled datasets. Other issues were present, such as the SHAP library being incompatible with RoBERTa layers. SHAP is a model interpretability library used to calculate statistics about the model predictions, and came before Captum.

7. Interpretation and Conclusions

The study was guided by the following objectives:

- I. Review existing literature on utilising LLMs to detect AI generated text within large datasets with a focus on phishing emails and the associated dataset construction.
- II. Analyse the baseline performance of the RoBERTa model across datasets with varying degrees of difficulty.
- III. Develop an optimised RoBERTa model through iterative training on differing datasets and verify its performance through the process whilst providing insights on what features affect the performance.

7.1 Conclusions

This dissertation attempted to answer the following questions:

1. What is the baseline performance of an untrained RoBERTa model across varying datasets?

The token impact bar graphs for the base model show that the base model was putting a large amount of emphasis on punctuation and other special characters rather than words. Since the base model was trained on Wikipedia content in addition to books and other non-email text it did not have much knowledge in what words often appear in a phishing email. The accuracy/confidence hovering around 50% means that the base model was essentially guessing at what type of email the sample was. This is where the RoBERTa model needed fine tuning to learn the unique language within emails. For example, emails will inherently be more informal in context and content than what the model has been accustomed to. It could be determined that the base model does not have a defensive posture to identify non valid emails. It can be seen that this is developed as the training progresses.

2. Can the RoBERTa model be easily optimised to classify emails as Phishing?

In the bar graphs produced for the model after the final stage of training, confidence in the prediction for the chosen samples increased significantly, and the fine-tuned model correctly identified the Spam and Valid emails for the chosen samples where the base model could not. While overall accuracy dropped significantly in stage 4 due to inequalities in the training datasets that had been used to initially fine tune the model, it

outperformed the base model in a direct comparison on the same selection of comparison samples.

Attaining a broad spectrum of modern email datasets was difficult – hence the research of (Toth, Gruschka and Bisztray, 2026) to develop a dataset. As discussed within the literature review section, modern phishing attacks are becoming ever more sophisticated with the development in LLMs. This directly leads to an arms race to acquire datasets that are modern so new threats can be picked up. This has happened for decades within an Electronic Warfare domain where new threats need to be known so they can be mitigated. Cyber warfare is a modern domain where data is king.

With access to datasets of increasing difficulty, I think the accuracy would not decrease as much when reaching the PhishFuzzer dataset. The datasets that were used to initially train the model were lacking in all classes. I believe that this hindered the model in understanding the differential and intent between a phishing email and a spam email.

Overall, I conclude that the RoBERTa model can be easily optimised to classify emails as Phishing or Valid. However, I would add that choosing the right datasets to use, as well as thorough data preparation, is vital to producing a well-optimised model.

3. What factors influence the accuracy of the model?

Several key factors of the training process affect the resulting model accuracy, such as dataset size, email type distribution, and the difference in email writing patterns between datasets. Dataset size affects accuracy because models need sufficient examples to learn patterns. With more samples, the model encounters more diverse variations of writing styles used in both phishing and valid emails, building generalized knowledge rather than being restricted to a smaller set of potentially similar emails. Email type distribution affects accuracy because the model's loss function weights all classes equally by default, so predicting the majority class is "cheaper" (less penalty) than predicting minority classes, incentivizing the model to bias toward common classes. The extremity of the difference in email writing patterns also affects accuracy, because if patterns are too different then the model will struggle to identify ones it knows already.

4. Are there limitations when trying to train RoBERTa using everyday computing power?

There were several limitations when using everyday and readily available computing power such as Colab. The primary issue was that insufficient processing power on the hardware used to access Colab caused frequent time-out issues. In addition, the free tokens provided by Colab to use with runtimes ran out quickly when using the most powerful GPU runtime (Nvidia A100). Without using this runtime, the project would not

have been deliverable in time for the project deadline. The cost of extra tokens on top of the subscription was not anticipated but was required to complete the project to a viable delivery state.

7.2 Implications

There are several implications as a consequence of the results observed and conclusions drawn from the development and analysis of the final model. One implication is that machine learning and model optimisation can be performed using everyday hardware and computational power. However, this comes with the trade off of having shorter times allowances for good quality training and fine-tuning. An additional disadvantage would be the selection of existing datasets, with most email datasets having the entire email grouped into one field, and the phishing/spam type label being the other. This would not provide good quality training as the model would potentially have trouble distinguishing between the body text and other sections of the email.

Another implication is that low-cost machine learning work focusing on phishing emails could have practical applications for related security software such as Airbus' Orion Malware detection tool. This tool resides on servers, and scans files for malware. If implemented securely and safely, AIML could be used to scan emails as well.

8. Recommendations and Further Work

While I am somewhat satisfied with the outcome of this project, there are several improvements and areas where I wish I had the time to explore Ai and machine learning deeper. One area for improvement is the amount of explainability graphs that were produced. I feel that in order to improve the comparison between the base model and my trained version, charts should have been produced at the end of each stage. This would have added significant further depth to the results of the training.

Another area of improvement was the intended implementation of Shap for the explainability metrics. I had originally intended to use Shap instead of Captum but failed to resolve difficulties with Shap's incompatibility with RoBERTa-based models. If I had more time to complete the project, I would have spent more time attempting to resolve this. Overall, the limited time I had to complete the project meant that it was not implemented to the depth and quality it could have been.

An area for future research is creating a custom labelled dataset structured like the PhishFuzzer dataset but incorporating emails from other datasets available online on sites such as HuggingFace. This would provide a bigger and richer dataset with proper structure that would enrich the training process.

9. References

- Abdali, S., Anarfi, R., Barberan, C. and He, J. (2024). *Decoding the AI Pen: Techniques and Challenges in Detecting AI-Generated Text*. [online] Available at: <https://arxiv.org/pdf/2403.05750v1> [Accessed 24 Oct. 2025].
- baeldung (2020). *Baeldung*. [online] Baeldung on Computer Science. Available at: <https://www.baeldung.com/cs/learning-curve-ml> [Accessed 3 May 2026].
- Bauskar, S.R., Madhavaram, C., Galla, E.P. and Gollangi, H.K. (2024). AI-Driven Phishing Email Detection: Leveraging Big Data Analytics for Enhanced Cybersecurity. *SSRN Electronic Journal*. doi:<https://doi.org/10.2139/ssrn.4980647>.
- Cisco (2025). *Cybersecurity Readiness Index 2025*. [online] cisco. Available at: <https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2025/m05/cybersecurity-readiness-index-2025.html>.
- Fraser, K.C., Dawkins, H. and Kiritchenko, S. (2025). Detecting AI-Generated Text: Factors Influencing Detectability with Current Methods. *Journal of Artificial Intelligence Research*, 82, pp.2233–2278. doi:<https://doi.org/10.1613/jair.1.16665>.
- Google for Developers (2026a). *Classification: Accuracy, recall, precision, and related metrics*. [online] Google for Developers. Available at: https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall#recall_or_true_positive_rate [Accessed 8 May 2026].
- Google for Developers (2026b). *Thresholds and the confusion matrix*. [online] Google for Developers. Available at: <https://developers.google.com/machine-learning/crash-course/classification/thresholding> [Accessed 8 May 2026].
- IBM (2025). *Cost of a Data Breach Report 2025*. [online] IBM. Available at: <https://www.ibm.com/reports/data-breach>.
- Kaspersky (2026). *AI phishing: Why Scams Look Real - and How to Avoid Them*. [online] Kaspersky. Available at: <https://www.kaspersky.com/resource-center/threats/ai-phishing-scams>.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V. and Allen, P. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. [online] Available at: <https://arxiv.org/pdf/1907.11692>.
- Milmo, D. (2024). UK Engineering Firm Arup Falls Victim to £20m Deepfake Scam. *The Guardian*. [online] 17 May. Available at:

<https://www.theguardian.com/technology/article/2024/may/17/uk-engineering-arup-deepfake-scam-hong-kong-ai-video>.

Opara, C., Modesti, P. and Golightly, L. (2025). Evaluating Spam Filters and Stylometric Detection of AI-generated Phishing Emails. *Expert Systems with Applications*, [online] 276, p.127044. doi:<https://doi.org/10.1016/j.eswa.2025.127044>.

Perri, D., Simonetti, M. and Gervasi, O. (2021). Synthetic Data Generation to Speed-Up the Object Recognition Pipeline. *Electronics*, 11(1), p.2. doi:<https://doi.org/10.3390/electronics11010002>.

Sadasivan, V., Kumar, A., Balasubramanian, S., Wang, W. and Feizi, S. (2024). *Can AI-Generated Text Be Reliably Detected?* [online] *arXiv*. Available at: <https://arxiv.org/pdf/2303.11156>.

Talos (2025). *Cisco Talos 2024 Year in Review*. [online] Cisco Talos Blog. Available at: <https://blog.talosintelligence.com/2024yearinreview/>.

Toth, R., Gruschka, N. and Bisztray, T. (2026). *The Phish, The Spam, and The Valid: Generating Feature-Rich Emails for Benchmarking LLMs*. [online] *arXiv*. Available at: <https://arxiv.org/pdf/2511.21448v5#page=1.27> [Accessed 23 Mar. 2026].